



EJERCICIO: CONFIGURACIÓN Y DEPURACIÓN DE UN PROYECTO PHP CON VISUAL STUDIO CODE

1. OBJETIVO DEL EJERCICIO

En este ejercicio se configurará un entorno de desarrollo PHP utilizando Visual Studio Code, PHP, Xdebug y XAMPP.

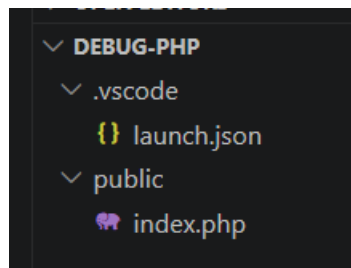
Se comprobarán dos formas diferentes de ejecutar el mismo proyecto:

- Servidor integrado de PHP
 - Visual Studio Code
 - PHP Built-in Server
 - <http://localhost:8000/>
- Apache mediante Virtual Host
 - XAMPP
 - Apache
 - Virtual Host
 - <http://php-mvc.localhost/>

Finalmente se utilizará Xdebug para colocar puntos de interrupción (*breakpoints*) y observar cómo se ejecuta el código PHP paso a paso.

2. ESTRUCTURA FINAL DEL PROYECTO

El proyecto tendrá la siguiente estructura:



La carpeta public será la carpeta pública de nuestra aplicación. Esto significa que el navegador tendrá acceso a los archivos que se encuentren dentro de public.

3. CREAR EL PROYECTO

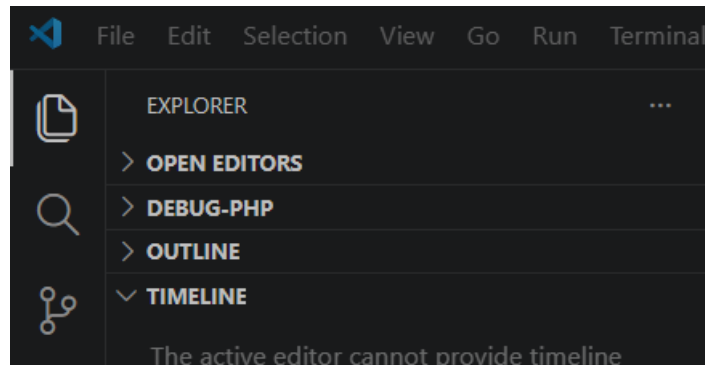
Crear la siguiente carpeta:

- C:\xampp\htdocs\debug-php

Dentro crear:

- C:\xampp\htdocs\debug-php\public

Abrir la carpeta **C:\xampp\htdocs\debug-php** con Visual Studio Code.



4. CREAR EL PROYECTO

Dentro de public, crear:

- index.php

Utilizaremos un programa sencillo que permita comprobar posteriormente el funcionamiento del debugger.

```
<?php

$nombre = "Marta";
$edad = 25;
function calcularEdadFutura(int $edad, int $anos)
{
    $resultado = $edad + $anos;
    return $resultado;
}

$edadFutura = calcularEdadFutura($edad, 5);
echo "<h1>Aplicación PHP de prueba</h1>";
echo "<p>Nombre: " . $nombre . "</p>";
echo "<p>Edad actual: " . $edad . " años.</p>";
echo "<p>Dentro de 5 años tendrás: " . $edadFutura . " años.</p>";

if ($edadFutura >= 30) {
    echo "<p>Ya tendrás 30 años o más.</p>";
} else {
    echo "<p>Todavía tendrás menos de 30 años.</p>";
}
?>
```

```
1  <?php
2
3  $nombre = "Marta";
4  $edad = 25;
5
6  function calcularEdadFutura(int $edad, int $años)
7  {
8      $resultado = $edad + $años;
9
10     return $resultado;
11 }
12
13 $edadFutura = calcularEdadFutura($edad, 5);
14
15 echo "<h1>Aplicación PHP de prueba</h1>";
16
17 echo "<p>Nombre: " . $nombre . "</p>";
18 echo "<p>Edad actual: " . $edad . " años.</p>";
19 echo "<p>Dentro de 5 años tendrás: " . $edadFutura . " años.</p>";
20
21 if ($edadFutura >= 30) {
22     echo "<p>Ya tendrás 30 años o más.</p>";
23 } else {
24     echo "<p>Todavía tendrás menos de 30 años.</p>";
25 }
26 ?>
27
```

5. COMPROBAR QUE PHP ESTÁ INSTALADO

Abrir una terminal y ejecutar:

- `php --version`

Debe aparecer la versión de PHP instalada.

```
C:\xampp\htdocs\debug-php>php --version
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies
    with Xdebug v3.5.3, Copyright (c) 2002-2026, by Derick Rethans

C:\xampp\htdocs\debug-php>
```

Si no aparece y sale el siguiente error significa que **Windows no encuentra PHP**.

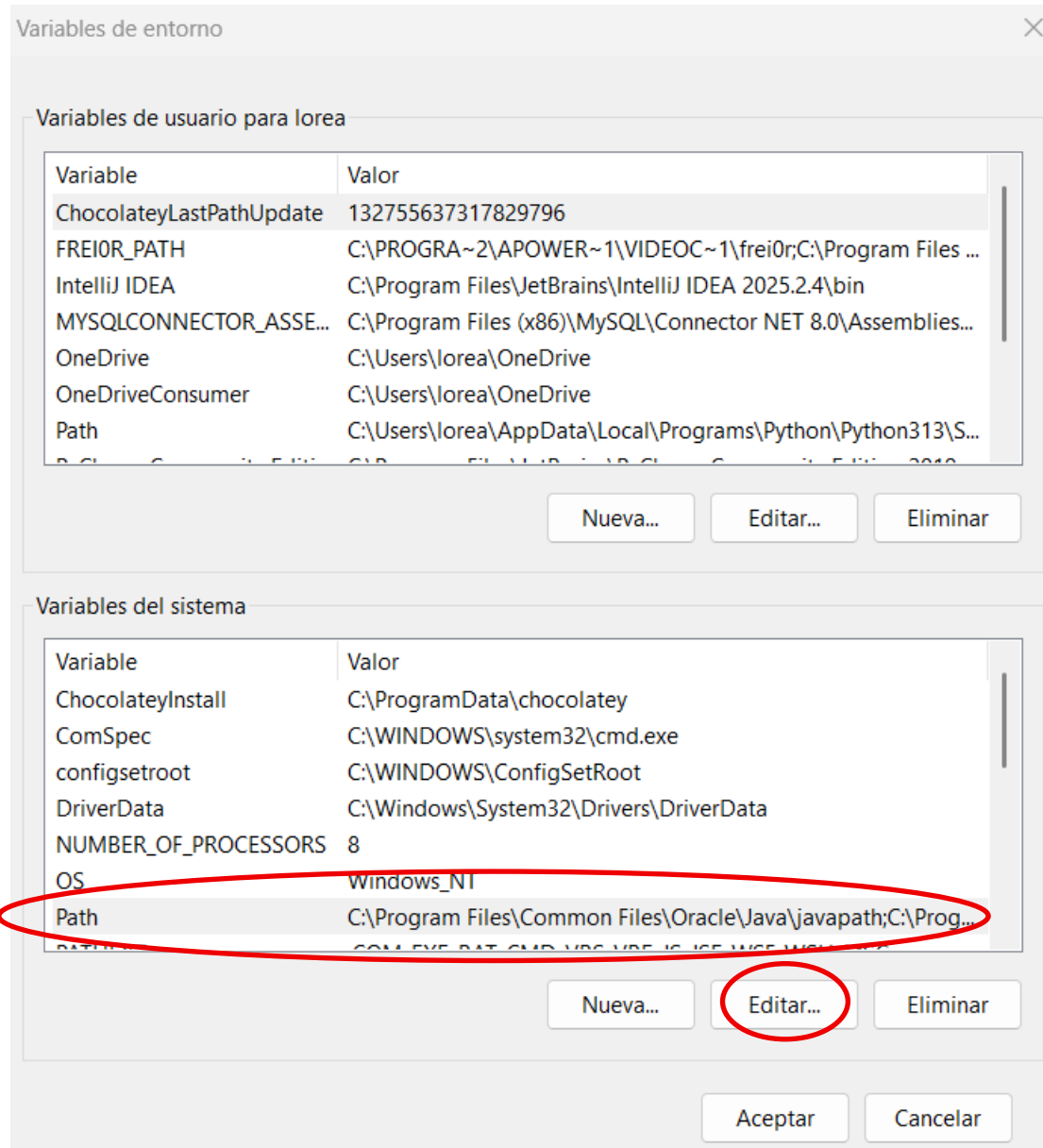
- "php" no se reconoce como un comando interno o externo, programa o archivo por lotes ejecutable.

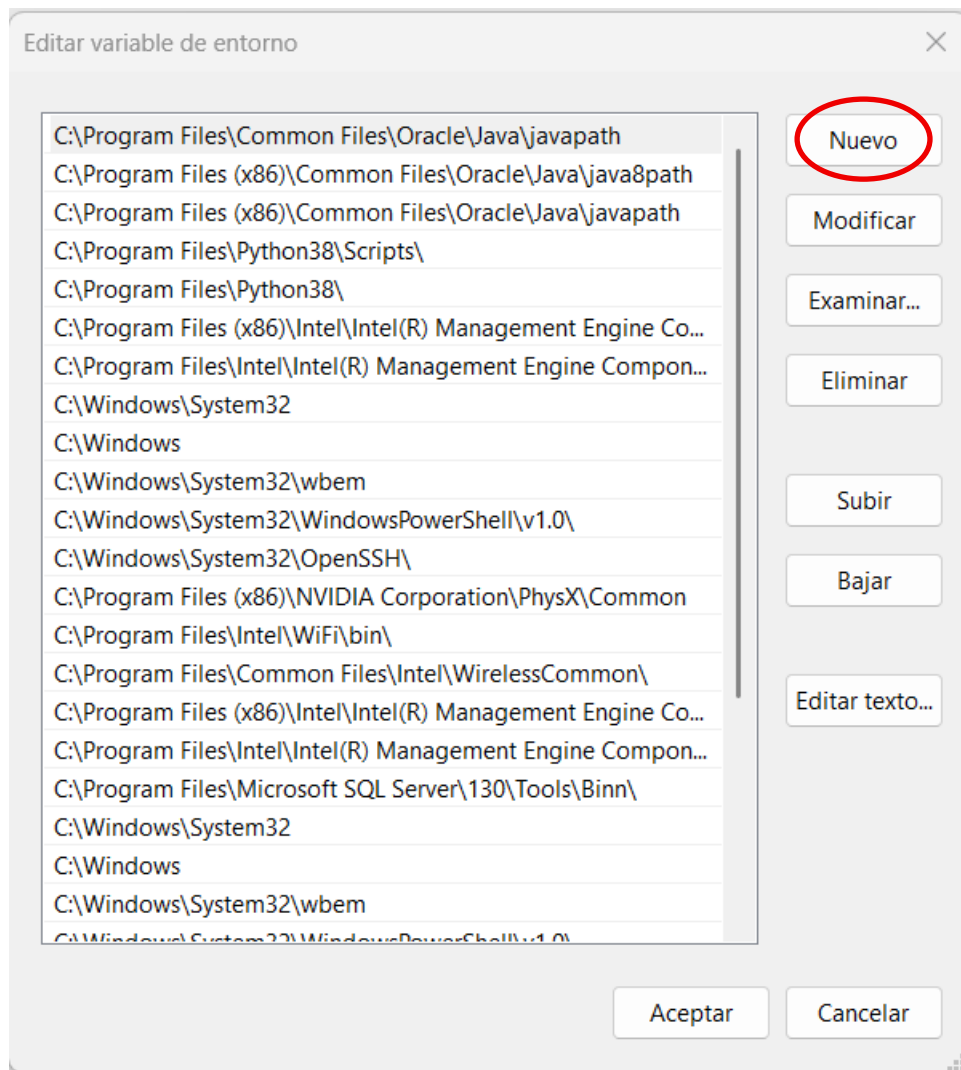
Solución:

Añadir PHP al PATH

Abrir en Windows:

- Inicio - Buscar "Variables de entorno"





Añadir:

- C:\xampp\php



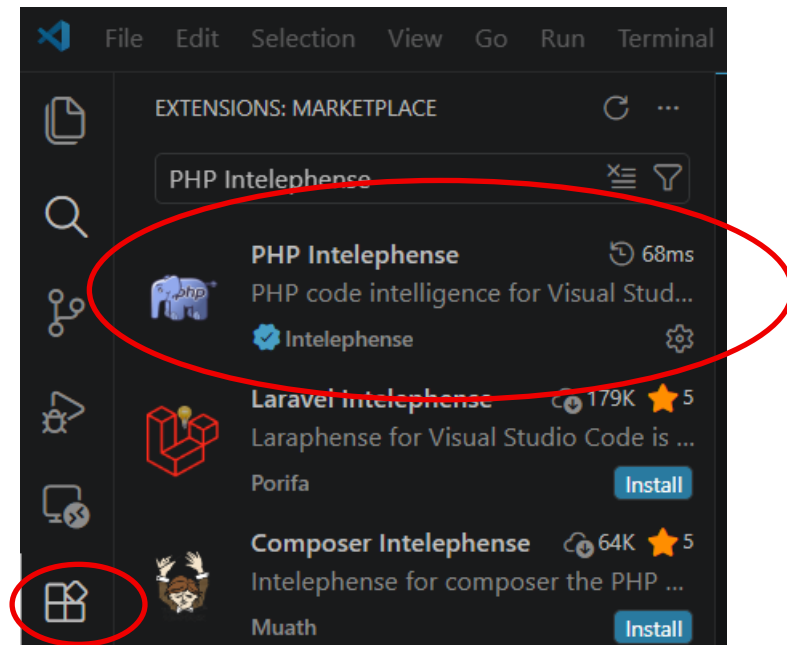
Aceptar todas las ventanas y volver a comporbar.

6. INSTALAR PHP INTELEPHENSE

Abrir:

- Extensions: PHP Intelephense

Instalar la extensión.



Intelephense proporciona asistencia específica para PHP:

- Autocompletado.
- Detección de errores.
- Información sobre funciones.
- Navegación entre clases.
- Información sobre parámetros.
- Análisis del código.

No ejecuta PHP ni sustituye a Xdebug. Su función principal es ayudarnos a escribir código PHP.

7. CREAR UN SNIPPET PHP

Un Snippet permite crear rápidamente código que utilizamos habitualmente.

Abrir la paleta de comandos:

- Ctrl + Shift + P

Escribir:

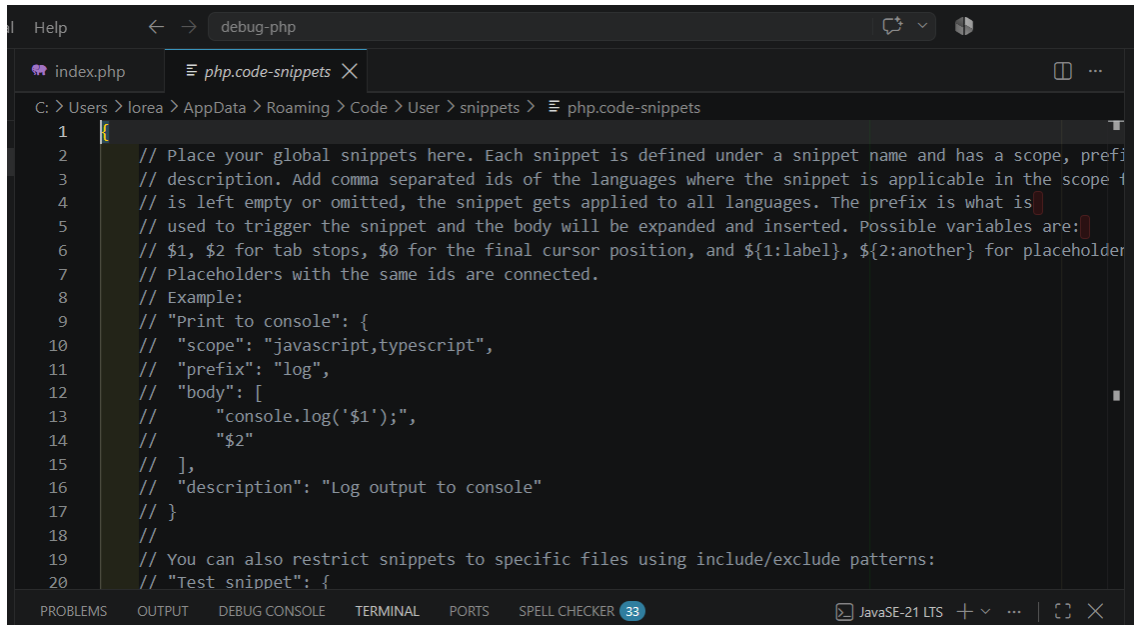
- Snippets

Seleccionar:

- Snippets: Configure Snippets
- New Global Snippets file...

Escribir:

- php.code-snippets

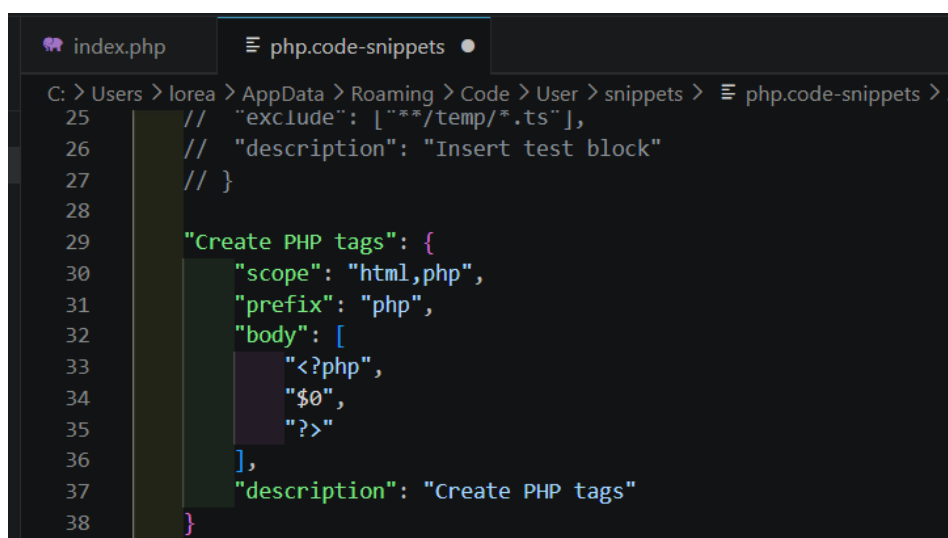


The screenshot shows the Visual Studio Code editor with the file `php.code-snippets` open. The file path is `C:\Users\lorea\AppData\Roaming\Code\User\snippets\php.code-snippets`. The content of the file is as follows:

```
1 {
2     // Place your global snippets here. Each snippet is defined under a snippet name and has a scope, prefix
3     // description. Add comma separated ids of the languages where the snippet is applicable in the scope
4     // is left empty or omitted, the snippet gets applied to all languages. The prefix is what is
5     // used to trigger the snippet and the body will be expanded and inserted. Possible variables are:
6     // $1, $2 for tab stops, $0 for the final cursor position, and ${1:label}, ${2:another} for placeholder
7     // Placeholders with the same ids are connected.
8     // Example:
9     // "Print to console": {
10    //   "scope": "javascript,typescript",
11    //   "prefix": "log",
12    //   "body": [
13    //     "console.log('$1');",
14    //     "$2"
15    //   ],
16    //   "description": "Log output to console"
17    // }
18    //
19    // You can also restrict snippets to specific files using include/exclude patterns:
20    // "Test snippet": {
```

Introducir:

```
"Create PHP tags": {
  "scope": "html,php",
  "prefix": "php",
  "body": [
    "<?php",
    "$0",
    "?>"
  ],
  "description": "Create PHP tags"
}
```



The screenshot shows the Visual Studio Code editor with the file `php.code-snippets` open. The file path is `C:\Users\lorea\AppData\Roaming\Code\User\snippets\php.code-snippets`. The content of the file is as follows:

```
25 // "exclude": [ "**/temp/*.ts" ],
26 // "description": "Insert test block"
27 // }
28
29 "Create PHP tags": {
30     "scope": "html,php",
31     "prefix": "php",
32     "body": [
33         "<?php",
34         "$0",
35         "?>"
36     ],
37     "description": "Create PHP tags"
38 }
```

En un archivo PHP podremos escribir:

- `php`

y seleccionar:

- `Create PHP tags`

generará:

`<?php`

`?>`

8. EJEMPLO PRÁCTICO DEL SNIPPET PHP

Una vez configurado el archivo `php.code-snippets`.

Paso 1. Crear un archivo PHP

Dentro de la carpeta `public` crea un archivo llamado:

- `prueba-snippets.php`

Al principio el archivo estará vacío.

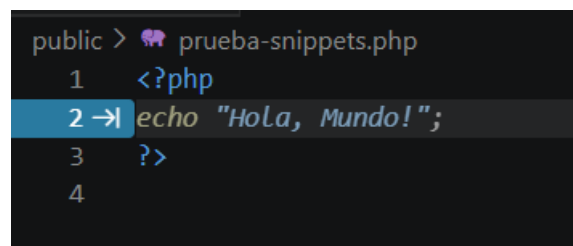
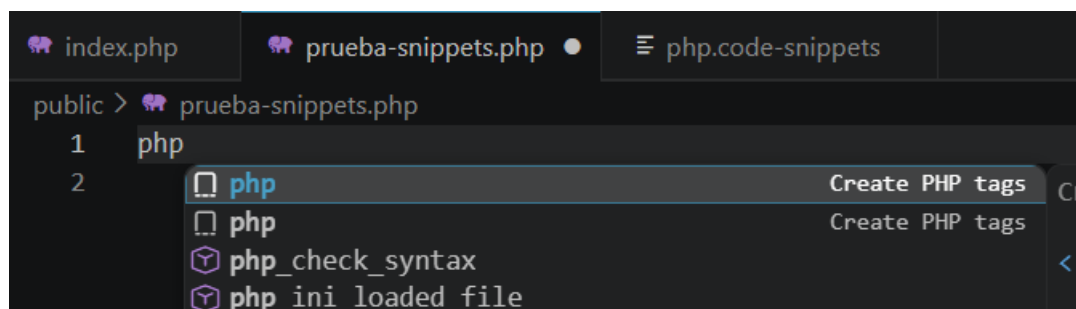
Paso 2. Escribir el prefijo del Snippet

Dentro de `snippets.php`, escribe solamente:

- `php`

No escribas todavía `<?php`.

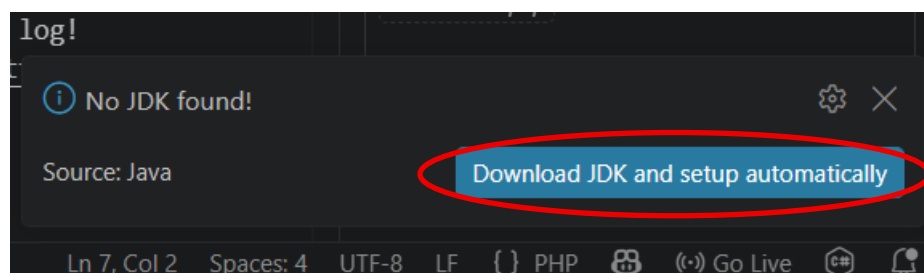
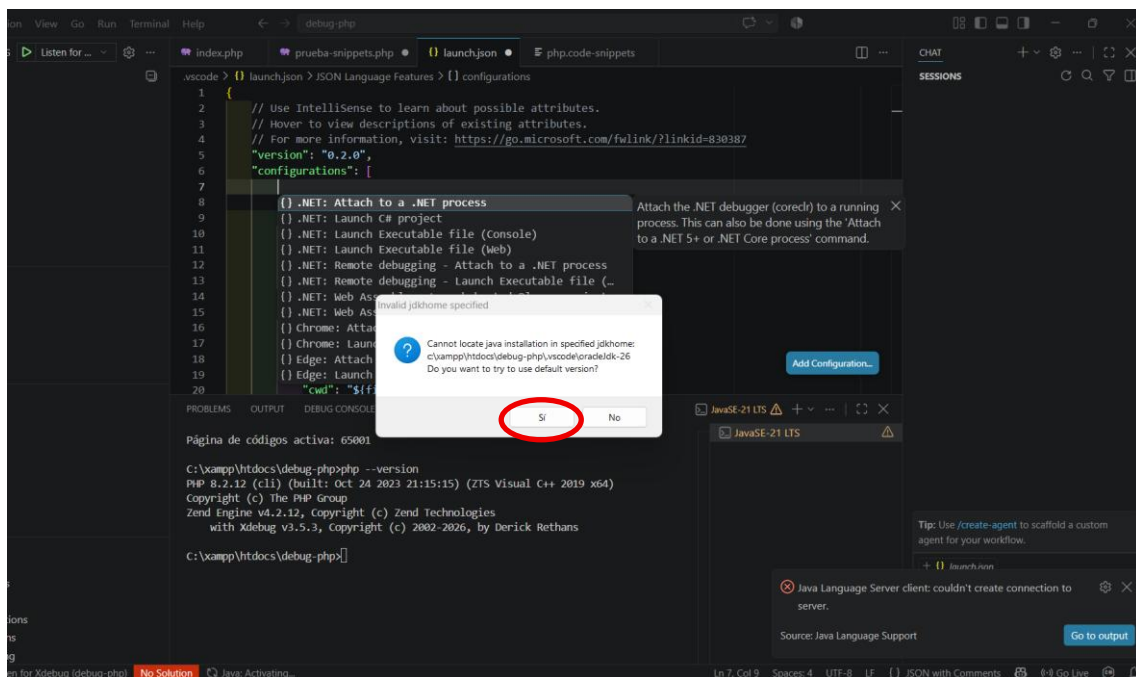
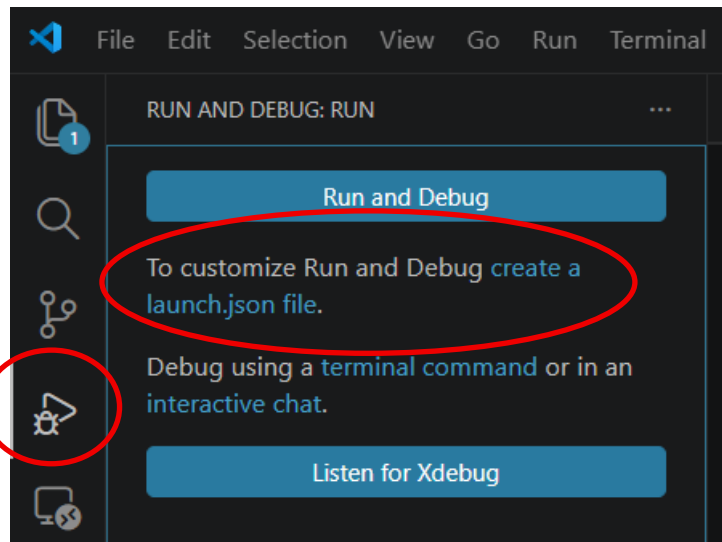
Visual Studio Code debería mostrar una sugerencia similar a:

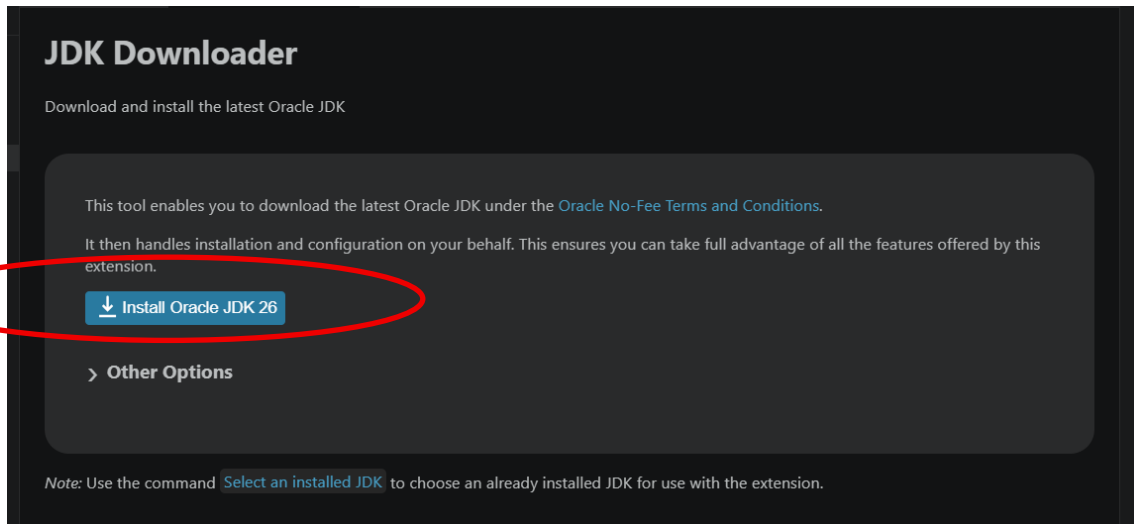


9. CONFIGURAR LAUNCH.JSON

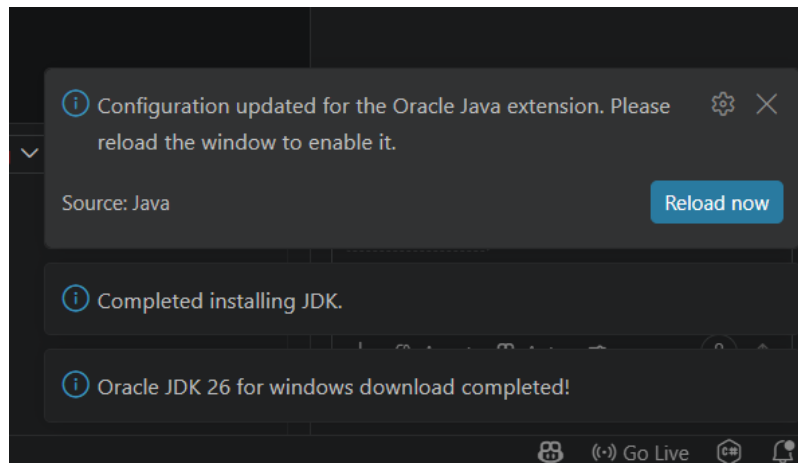
Dentro del proyecto crear:

- .vscode\launch.json





Instalar en C:\xampp\htdocs y reinicia Visual Code.



Utilizar la siguiente configuración:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Listen for Xdebug",
      "type": "php",
      "request": "launch",
      "port": 9003
    },
    {
      "name": "Launch Built-in web server",
      "type": "php",
      "request": "launch",
      "runtimeArgs": [
        "-S",
        "localhost:8000",
        "-t",
        "."
      ],
      "cwd": "${workspaceFolder}/public",
      "port": 9003,
      "serverReadyAction": {
        "action": "openExternally"
      }
    }
  ]
}
```

```
30 {
31     "name": "Launch Built-in web server",
32     "type": "php",
33     "request": "launch",
34     "runtimeArgs": [
35         "-S",
36         "localhost:8000",
37         "-t",
38         "."
39     ],
40     "cwd": "${workspaceFolder}/public",
41     "port": 9003,
42     "serverReadyAction": {
43         "action": "openExternally"
44     }
45 }
46 ]
47 ]
48 }
```

10. EJECUTAR CON PHP BUILT-IN SERVER

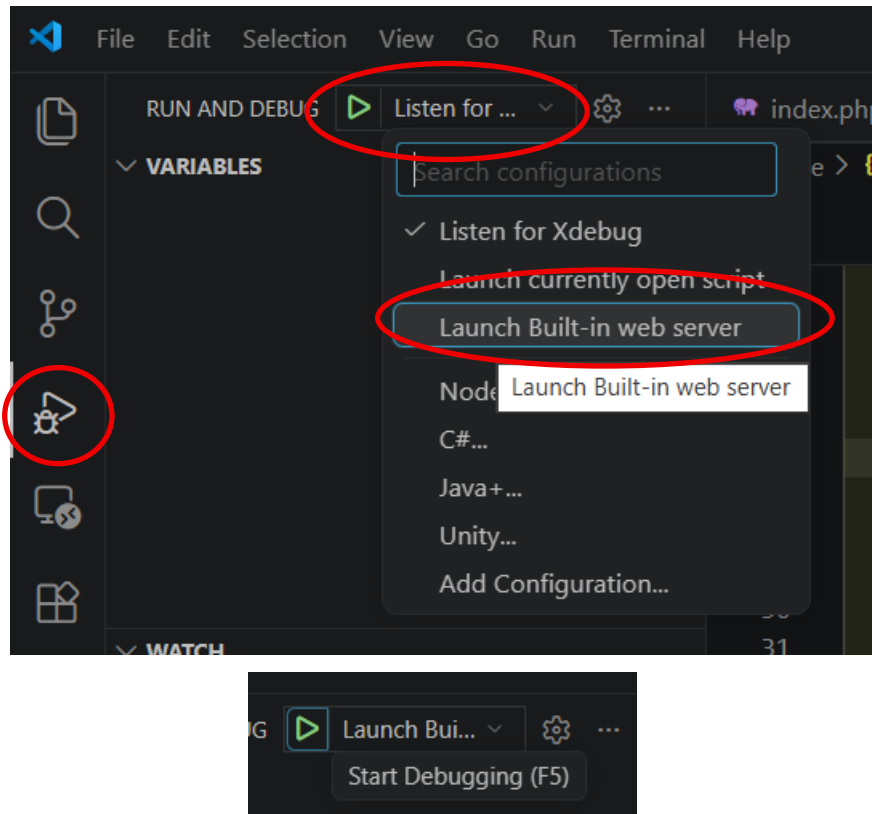
PHP Built-in Server: PHP incluye un pequeño servidor web que podemos utilizar durante el desarrollo. No necesitamos Apache para esta prueba. Visual Studio Code ejecutará internamente:

- `php -S localhost:8000`

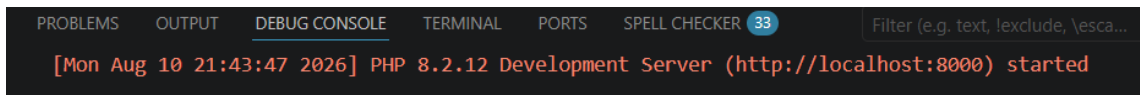
Por tanto, **localhost:8000** será nuestro servidor PHP.

En Visual Studio Code:

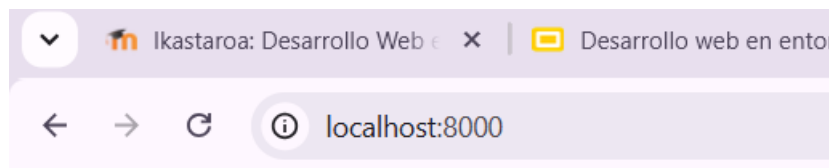
1. Abrir Run and Debug.
2. Seleccionar:
 - Launch Built-in web server
3. Pulsar:
 - Start Debugging(F5)



En la consola debería aparecer un mensaje similar a:



Comprobar desde el navegador:



Aplicación PHP de prueba

Nombre: Marta

Edad actual: 25 años.

Dentro de 5 años tendrás: 30 años.

Ya tendrás 30 años o más.

11. PRIMERA PRUEBA DE DEBUGGER

Abrir:

- `public/index.php`

Buscar:

- `$edadFutura = calcularEdadFutura($edad, 5);`

Colocar un breakpoint haciendo clic a la izquierda del número de línea.

Aparecerá un punto rojo.

Después iniciar, Run and Debug

- Launch Built-in web server

Actualizar:

<http://localhost:8000/>

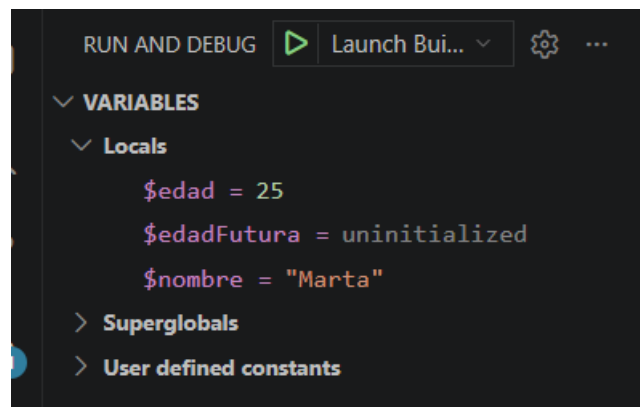
La ejecución debería detenerse en el breakpoint.

12. ANALIZAR LAS VARIABLES

Cuando el debugger se detenga, observar la sección:

VARIABLES

Deberíamos poder observar:



13. CONTROLES PRINCIPALES DEL DEBUGGER

Cuando el programa se detenga aparecerán diferentes botones.

Continue: Continúa la ejecución hasta el siguiente breakpoint.

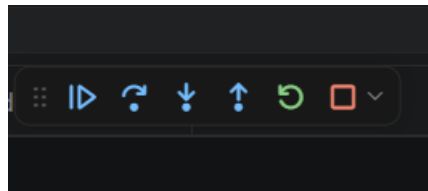
Step Over: Ejecuta la siguiente línea sin entrar en las funciones.

Step Into: Entra dentro de una función.

Step Out: Sale de la función actual y vuelve al punto desde donde fue llamada.

Restart: Reinicia la depuración.

Stop: Detiene la ejecución.



14. EJECUTAR CON APACHE Y VIRTUAL HOST

Ahora vamos a ejecutar el mismo proyecto, pero utilizando Apache.

La diferencia será, **PHP Built-in Server** <http://localhost:8000/> y con **Apache** <http://php-mvc.localhost/>

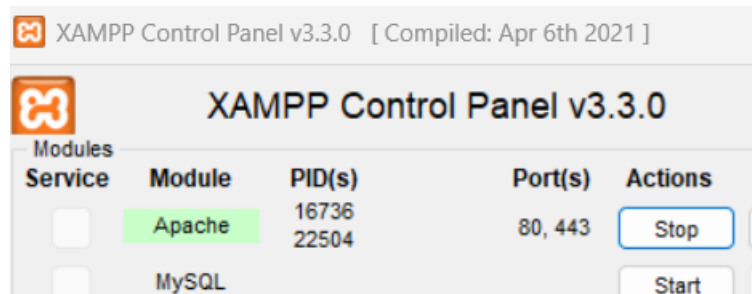
15. COMPROBAR APACHE

Abrir:

- XAMPP Control Panel

Iniciar:

- Apache



16. CONFIGURAR EL VIRTUAL HOST

Abrir:

- C:\xampp\apache\conf\extra\httpd-vhosts.conf

Añadir:

```
<VirtualHost *:80>
  ServerAdmin webmaster@localhost
  DocumentRoot "C:/xampp/htdocs/debug-php/public"
  ServerName php-mvc.localhost
  ErrorLog "logs/php-mvc-error.log"
  CustomLog "logs/php-mvc-access.log" common
</VirtualHost>
```

Con esto estamos indicando a Apache, cuando el navegador solicite **php-mvc.localhost**, debe utilizar como raíz la carpeta **public** de nuestro proyecto. Por tanto, <http://php-mvc.localhost/> apuntará a: **C:\xampp\htdocs\debug-php\public**

17. REINICIAR APACHE

Guardar:

- httpd-vhosts.conf

En XAMPP:

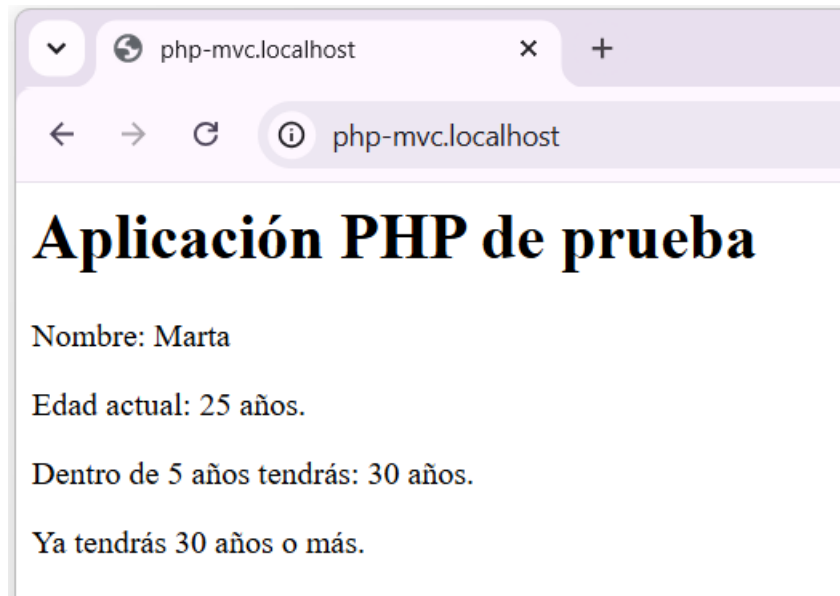
1. Stop Apache.
2. Start Apache.

18. COMPROBAR EL VIRTUAL HOST

Abrir:

- `http://php-mvc.localhost/`

Debe aparecer el mismo contenido que antes:



Aparece lo mismo porque estamos ejecutando el mismo archivo:

- `public/index.php`

Lo único que ha cambiado es el servidor que recibe la petición.

19. DEPURACIÓN UTILIZANDO XDEBUG

Ahora vamos a utilizar Xdebug para analizar la ejecución. Colocar un breakpoint en:

- `$edadFutura = calcularEdadFutura($edad, 5);`

La ejecución se detendrá en esta línea cuando la página sea solicitada.

20. DEPURAR EL PROYECTO MEDIANTE LOCALHOST:8000

Para probar la primera opción:

1. Seleccionar:
 - Launch Built-in web server
2. Iniciar el debugger.
3. Abrir:
 - <http://localhost:8000/>
4. El programa deberá detenerse en el breakpoint.
5. Analizar las variables.
6. Utilizar Step Over y Step Into.

25. DEPURAR EL PROYECTO MEDIANTE APACHE (HTTP://PHP-MVC.LOCALHOST/)

Para probar la segunda opción:

1. Iniciar Apache desde XAMPP.
2. Mantener abierto VS Code.
3. Seleccionar:
 - Listen for Xdebug
4. Pulsar Start Debugging.
5. Abrir:
 - <http://php-mvc.localhost/>
6. La ejecución debería detenerse en el breakpoint.

Importante

En este caso Apache es quien ejecuta PHP, mientras que Visual Studio Code permanece a la espera de la conexión de Xdebug.

El flujo es: Navegador – Apache – PHP – Xdebug - Visual Studio Code

26. CONCLUSIÓN

localhost:8000 utiliza el servidor integrado de PHP, mientras que **php-mvc.localhost** utiliza Apache mediante un Virtual Host. En ambos casos se ejecuta el mismo archivo **public/index.php** y Xdebug permite detener y analizar la ejecución del código PHP.